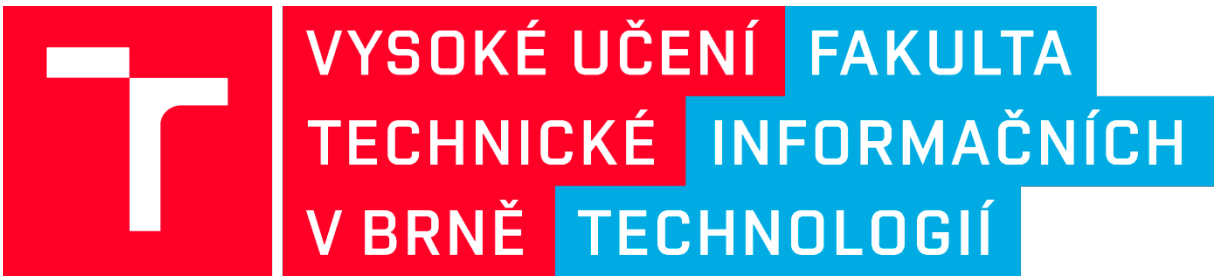


Dokumentácia implementácie projektu

Interpret objektového jazyka SOL26

Martin Turčan, xturcam00

1. Celkový popis návrhu

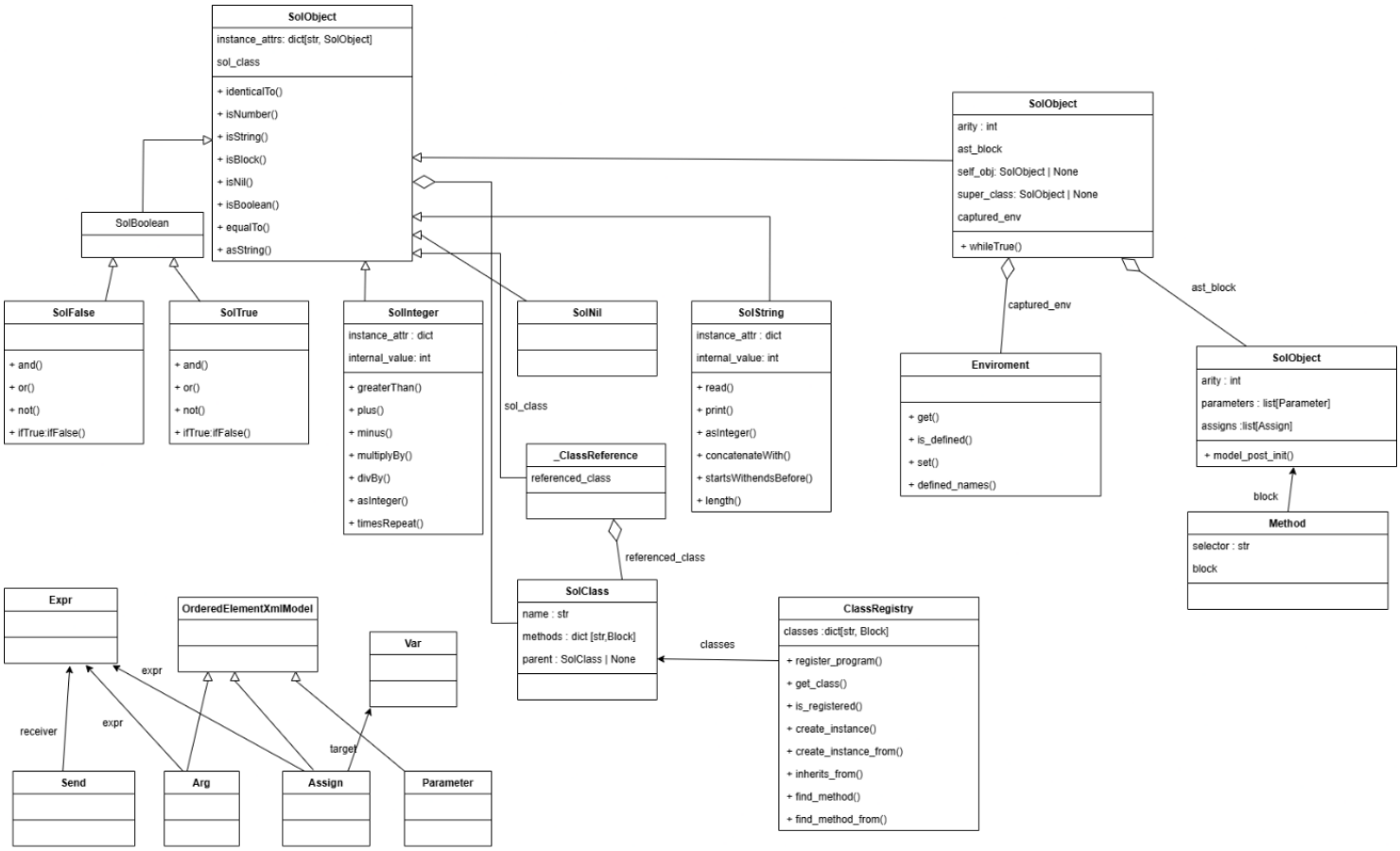
Implementácia je rozdelená do samostatných vrstiev, ktoré sú zodpovedné za jednotlivé fázy behu interpretu: vstupná vrstva (solint.py), načítanie a validácia XML modelu (input_model.py, interpreter.py), registrácia tried a vytvorenie runtime modelu (class_registry.py, runtime/*) a samotné vyhodnocovanie programu (evaluator.py, builtins/*).

Toto rozdelenie oddeľuje statickú fázu (kontrola štruktúry a časti sémantiky) od dynamickej fázy (odosielanie správ, vyhodnocovanie blokov, práca s prostredím).

Runtime model je postavený na objektoch SolClass, SolObject a na ďalších objektoch ktoré dedia zo SolObject. ClassRegistry registruje vstavané aj užívateľské triedy, rieši vzťahy dedičnosti a poskytuje vyhľadávanie metód a vytváranie inštancií ‘new‘, ‘from‘. Objekty uchovávajú inštančné atribúty v ‘map-e‘ a metódy sú uložené na úrovni tried.

Interpretácia prebieha v troch funkciách: eval_expr (vyhodnotenie výrazu), execute_block (vykonanie sekvencie priradení v bloku) a send_message (odoslanie správy). Odosielanie správ používa poradie: užívateľská metóda, vstavaná metóda, atribút inštancie (getter/setter) a chyba DNU. Bloky sú reprezentované objektom SolBlock a pri vytvorení zachytávajú lexikálne prostredie a aj kontext self.

1.1 UML – Class Diagram



2. Popis interných dátových štruktúr

- **SolClass** – je runtime inštancia triedy obsahujúca názov, odkaz na nadradenú triedu a mapu metód. Slúži ako uzol dedičnej hierarchie pre vyhľadávanie metód aj vstavaných operácií.
- **SolObject + potomkovia -** Základná implementácia inštancie ktorá obsahuje odkaz na triedu a mapu atribútov inštancie. Konkrétne triedy (SolInteger,

SolString, SolBlock, singletony SolNil/SolTrue/SolFalse) pridávajú stav alebo redefinujú metódy.

- **ClassRegistry -** Centrálny register všetkých dostupných tried zabezpečuje registráciu užívateľských tried, riešenie rodičov, vytváranie inštancií a vyhľadávanie metód. Plní úlohu integračného bodu medzi statickým modelom a prostredím runtime.
- **Enviroment -** Implementuje lexikálny rozsah pre lokálne premenné a parametre blokov. Vyhľadávanie aj zápis prebiehajú cez reťazec nadradených rámcov a formálne parametre sú chránené proti prepísaniu.
- **BUILTIN_METHODS –** (dispatch tabuľka) Vnorená štruktúra - class_name, selector, funkcia, ktorá izoluje vstavané správanie od jadra evaluátora. Umožňuje jednoduché pridávanie metód bez zásahov do základného dispatch mechanizmu.

3. Návrhové vzory a principy OOP

3.1 Registry Pattern

ClassRegistry centralizuje správu tried a nahrádza rozptýlené globálne vyhľadávania. V jednom objekte sa nachádza registrácia vstavaných tried, načítanie užívateľských tried, riešenie rodičovských tried, vyhľadávanie metód a metódy create_instance/create_instance_from. Výhodou je konzistentnosť behu, jednoduchá validácia redefinícií a jasná integračná hranica medzi parserom a evaluátorom. Rozširiteľnosť je vysoká pretože nové triedy, pravidlá

vyhľadávania alebo konštrukčná logika sa menia primárne tu.

3.2 Interpreter pattern (AST - vyhodnocovanie)

Funkcia eval_expr vykonáva vyhodnotenie AST podľa typu uzla, zatiaľ čo execute_block interpretuje sekvenciu príkazov a vráti hodnotu posledného výrazu. Tento štýl priamo premieta sémantiku jazyka do vykonávacieho mechanizmu a udržiava logiku behu v jednej konzistentnej vrstve. V kombinácii s typizovaným modelom AST to znižuje počet neplatných stavov počas behu a umožňuje dobre čitateľný tok vyhodnocovania (literal → var → block literal → send).

3.3 Singleton pattern pre Nil/False/True

Triedy SolNil, SolFlase a SolTrue implementujú jedinú inštanciu objektu v rámci behu interpretu a registre túto vlastnosť pri konštrukcii rešpektujú. Tým je zaručená identická sémantika singletonov (nil, false, true) a zjednodušuje sa porovnávanie aj správanie booleovských/nil operácií.

4. Zásah do šablony

Šablóna bola rozšírená iba v jednom súbore a to v ‘input_model.py‘, kde začiatok súboru bolo nutné pridať –

from __future__ import annotations - z dôvodu dopredných referencií.

5. Popis a riešenie najväčších problémov

5.1 Closure semantika a lexikálne scopes

Prepojiť beh blokov s lexikálnym prostredím tak, aby blok niesol zachytený kontext aj mimo miesta svojho vzniku. Riešenie používa SolBlock ktorý pri vykonaní vytvorí nový rámec s odkazom parent na zachytené prostredie. Tým je zabezpečená viditeľnosť premenných v lexikálne vnorených blokoch a zároveň ochrana formálnych parametrov proti prepísaniu.

5.2 Implementácia dispatch pravidiel

Najväčším problémom bolo správne poradie a podmienky odosielania, najmä v kombinácii s dedičnosťou a metódou super. Riešenie spočíva v centralizácii do send_message, kde je stanovené pevné poradie: užívateľská metóda, vstavaná metóda a až potom atribútový getter/setter. Tým sa minimalizujú nekonzistencie a dá sa ľahko zistiť, prečo konkrétna správa skončila chybou 51/54.

6. Rozšírenia

6.1 Mechanizmus doesNotUnderstand

Pri mojej implementácii by bolo najjednoduchšie implementovať mechanizmus doesNotUnderstand, pretože interpreter už má centralizovaný dispatch správ v send_message, kde sa dnes priamo vyhadzuje chyba 51 pri nenájdení metódy, a stačí teda zmeniť posledný krok fallbacku. Namiesto okamžného ukončenia sa na príjemcovi zavolá doesNotUnderstand: s argumentom nového objektu Message, ktorý zapuzdruje pôvodný selektor a argumenty správy; trieda Object dostane predvolenú implementáciu doesNotUnderstand:, ktorá zachová súčasné správanie (vyvolanie chyby 51), zatiaľ čo používateľské triedy ju budú môcť predefinovať a implementovať vlastné dynamické spracovanie neznámych správ.

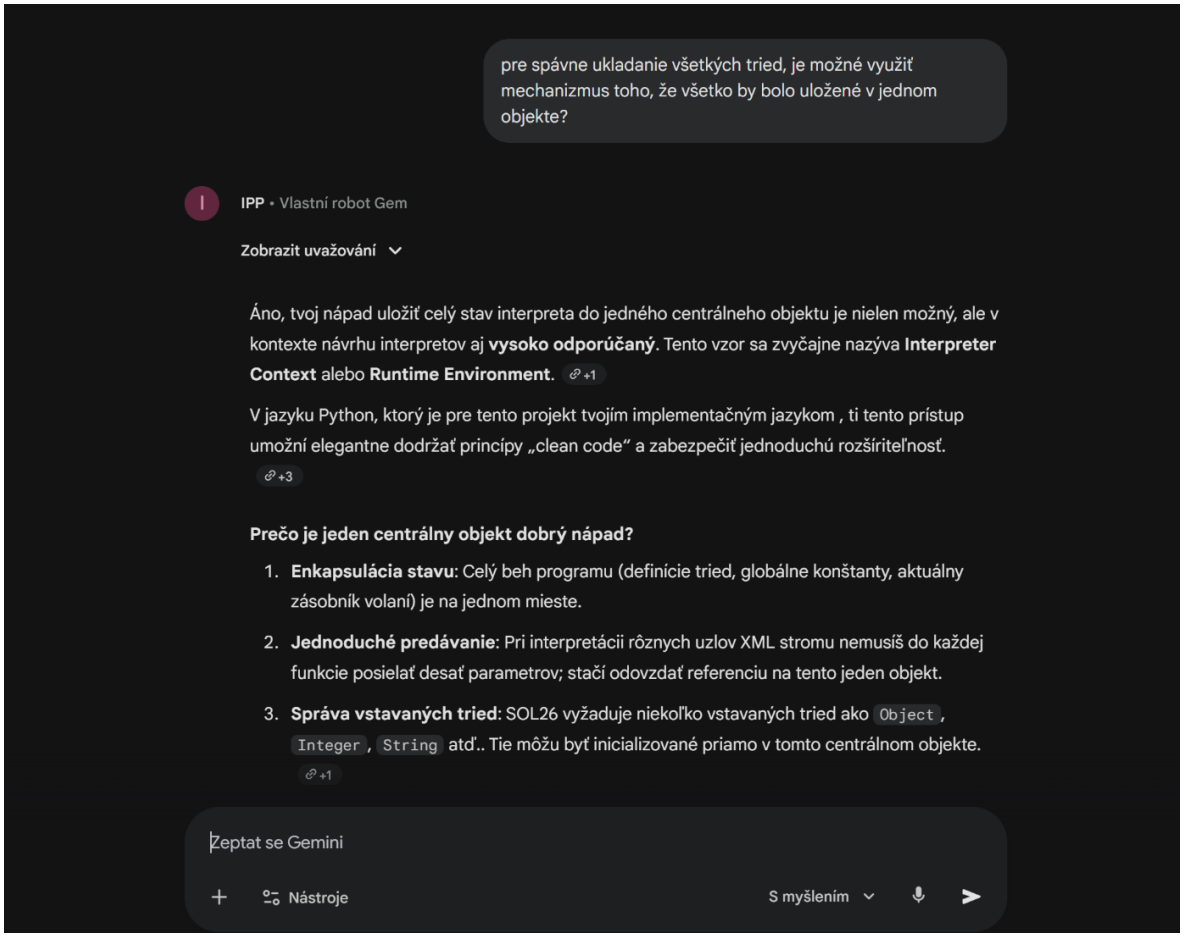
6.2 Ladiaci režim s krokovaním

Ladiaci režim s krokovaním sa nadstavuje nad už existujúci vykonávaný tok v execute_block(), kde interpreter prirodzene prechádza jednotlivé príkazy priradenia. Implementácia spočíva v doplnení CLI prepínača --debug v solint.py, prenesení debug príznaku do runtime kontextu cez ClassRegistry a následnom vypisovaní diagnostiky pred každým assign na stderr (kontext trieda/metóda/blok, poradie príkazu, cieľová premenná, prípadne hodnota výrazu po vyhodnotení). Pri breakpointoch napr. cez XML atribút breakpoint="true" interpreter vstúpi do jednoduchého interaktívneho promptu so základnými

príkazmi step, continue, vars a self, čím sa umožní priebežná inšpekcia stavu bez zmeny samotného programu.

7. Využívanie AI nástrojov

Pri používaní AI nástrojov som používal najmä 2 veľké jazykové modely Geminy a Claude. Geminy som používal na vysvetlenie zadania, nejasností a problémov ktoré nastali. Pre správne použitie som kládol dôraz na to aby prompty boli presné a aby neobsahovali mylné a duplicitné informácie. Claude mi bol nápomocný pri tvorbe programov na testovanie v Sol26 a to tak, že som mu dal analyzovať špecifikácie jazyku a následne podľa toho vytvoril programy na testovanie, ktoré som musel prejsť a validovať či sú správne a testujú to čo naozaj majú. Ako poslednú vec som AI použil na štylistiku a úpravu gramatiky textu, aby dokumentácia bola stručná, výstižná a gramaticky správna.



Si odborník na písanie dockerov a ich vysvetlovanie popíš mi čo je to docker a ako funguje

Čo je Docker a ako funguje?

🐳 Základná myšlienka

Docker je nástroj na **kontajnerizáciu aplikácií**. Zjednodušene povedané — zabalí tvoju aplikáciu spolu so všetkým, čo potrebuje (knižnice, závislosti, konfigurácia), do jedného prenosného "balíčka" nazývaného **kontajner**.

> **Analógia:** Predstav si lodný kontajner. Nezáleží, či ho prepravuješ loďou, vlakom alebo kamiónom — obsah zostane rovnaký. Docker funguje rovnako, len pre softvér.

🧱 Kľúčové pojmy

Image (obraz)

- Šablóna/recept na vytvorenie kontajnera
- Je nemenný (read-only)
- Napríklad: `ubuntu:22.04`, `node:20`, `postgres:16`

Kontajner

- Bežiaca inštancia image-u
- Izolovaný proces na tvojom systéme
- Z jedného image môžeš spustiť desiatky kontajnerov

Dockerfile

- Textový súbor s inštrukciami na zostavenie image-u
- Každý riadok = jedna vrstva (layer)

Docker Hub / Registry

- Úložisko hotových image-ov (ako GitHub, ale pre Docker image-y)

⚙️ Ako to funguje pod kapotou?

Docker využíva funkcie **Linux kernelu:**

- **Namespaces** — izolujú kontajner (vlastný filesystem, sieť, procesy)
- **cgroups** — limitujú zdroje (CPU, RAM, disk)
- **Union filesystem** — vrstvy image-u sa skladajú na seba efektívne

```
┌──────────────────────────────┐
│        Tvoja aplikácia       │
├──────────────────────────────┤
│     Závislosti / Knižnice    │
├──────────────────────────────┤
│        Base image (OS)       │
├──────────────────────────────┤
│        Docker Engine         │
├──────────────────────────────┤
│   Host OS (Linux/Win/macOS)  │
└──────────────────────────────┘
```

📄 Príklad jednoduchého Dockerfile-u

```dockerfile
# 1. Začíname od oficiálneho Node.js image-u
FROM node:20-alpine

# 2. Nastavíme pracovný adresár v kontajneri
WORKDIR /app

# 3. Skopírujeme závislosti a nainštalujeme ich
COPY package*.json ./
RUN npm install

# 4. Skopírujeme zvyšok kódu
COPY . .

# 5. Otvoríme port 3000
EXPOSE 3000

# 6. Spustíme aplikáciu
CMD ["node", "server.js"]
```

Zostavenie a spustenie:

```bash
docker build -t moja-aplikacia .   # vytvorí image
docker run -p 3000:3000 moja-aplikacia  # spustí kontajner
```

🆚 Docker vs. Virtuálny stroj (VM)

	Docker	VM
Štart	Sekundy	Minúty
Veľkosť	MB	GB
Izolácia	Proces	Celý OS
Zdieľa kernel	✅ Áno	❌ Nie
Bezpečnosť	Menšia izolácia	Väčšia izolácia

🎯 Prečo ho používať?

- **"U mňa to funguje"** — kontajner beží rovnako všade
- **Rýchle nasadenie** — spustíš prostredie jedným príkazom
- **Škálovateľnosť** — ľahko spustíš viac inštancií
- **Čistota systému** — závislosti sú izolované, nič sa nemieša

Gemini

Analýza projektu a plán implementácie

Upgradujte na Google AI Plus

M

Výskum a analýza jazyka SOL26

Prehľadávanie webov

(1) Vyhľadať informácie o jazyku Smalltalk a jeho objektovom modeli, aby sa lepšie pochopila filozofia, ktorou je jazyk SOL26 inšpirovaný.
(2) V súbore ipp26.pdf identifikovať kľúčové termíny projektu, povolené kombinácie programovacích jazykov (Python, TypeScript, PHP) a termíny odevzdania.
(3) V dokumente sol26-spec.pdf preskúmať definíciu jazyka SOL26, jeho lexikálne jednotky, identifikátory a pravidlá pre case-sensitivity.
(4) Podrobne analyzovať sémantiku jazyka SOL26 v sol26-spec.pdf, so zameraním na zasielanie správ, prácu s blokmi ako objektmi a podporu uzáverov (closures).
(5) V sol26-spec.pdf identifikovať mechanizmy dedičnosti, používanie kľúčových slov self a super, a pravidlá pre vytváranie inštančných atribútov.
(6) Preštudovať špecifikáciu XML reprezentácie programu (SOL-XML) v sol26-spec.pdf a zistiť, ako sú v nej mapované príkazy a bloky.
(7) V ipp26.pdf vyhľadať požiadavky na implementáciu testovacieho nástroja, štruktúru testovacích súborov (.test, .in, .out) a formát JSON reportu.
(8) Analyzovať požiadavky na kontejnerizáciu v ipp26.pdf, konkrétne stupne Containerfile (check, build, runtime, test) a povinné nástroje pre kontrolu kvality kódu.

Analýza výsledkov

Vytváranie reportu

O pár minút bude hotovo

Co chcete prozkoumat?

S myšlením

Gemini je AI a může dělat chyby (i co se týče lidí). Vaše soukromí a Gemini

Výskum a analýza jazyka SOL26

Obsah

Sdílet a exportovat

Vytvořit

Architektúra a implementácia objektovo orientovaného jazyka SOL26

Rozvoj moderných programovacích paradigiem je často ukotvený v historických konceptoch, ktoré definovali základné princípy interakcie medzi dátami a logikou. Jazyk SOL26, vyvinutý pre potreby projektu IPP 2025/2026, predstavuje syntézu imperatívnych, netypovaných a čisto objektovo orientovaných prístupov, ktoré nachádzajú svoju inšpiráciu v prelomovom jazyku Smalltalk. Tento dokument poskytuje vyčerpávajúcu technickú analýzu špecifikácie jazyka SOL26, jeho implementačného rámca a širšieho ekosystému nástrojov, ktoré sú nevyhnutné pre vytvorenie robustného interpretu v prostredí moderného softvérového inžinierstva.

Filozofické a historické základy objektového modelu

Pochopenie jazyka SOL26 si vyžaduje hlboký ponor do filozofie, ktorú v 70. rokoch 20. storočia sformovali Alan Kay, Adele Goldberg a Dan Ingalls v rámci výskumného strediska Xerox PARC. Smalltalk nebol len programovacím jazykom, ale celým výpočtovým prostredím založeným na metafore objektov, ktoré medzi sebou komunikujú výhradne prostredníctvom zasielania správ. SOL26 preberá túto základnú axiómu, kde každá hodnota, či už ide o celé číslo, reťazec alebo blok kódu, je inštanciou určitej triedy a správa sa ako autonómna entita.

V tradičných procedurálnych jazykoch je volanie funkcie statickým úkonom, kde o vykonávanom kóde rozhoduje prekladač alebo väzobný program. V Smalltalku a následne v SOL26 má však príjemca správy úplnú kontrolu nad tým, ako na danú správu zareaguje. Tento mechanizmus neskorého viazania (late binding) umožňuje vysokú mieru polymorfizmu a flexibility, čo je v SOL26 demonštrované skutočnosťou, že aj základné riadiace štruktúry, ako sú podmienky a cykly, sú implementované ako správy zasielané objektom tried True, False a Block.

Charakteristika	Implementácia v SOL26	Teoretický význam
Typový systém	Dynamický a netypovaný	Typ premennej je určený triedou referencovaného objektu v čase behu.
Model vykonávania	Interpretorom riadený AST	Interpret priamo vyhodnocuje abstraktný syntaktický strom v XML formáte.
Objektová hierarchia	Triedna dedičnosť	Podporuje definíciu tried, inštančných metód a atribútov s jednoduchou dedičnosťou.